

Limitations, Risks, and Deltas

Обновлено 29 мая 2026, 00:32 Андрей Коновалов

MVP Limitations

- Only one tournament mode: multi-round text prompt without elimination.
- Text submissions only.
- No spectator mode.
- No chat/comments/reactions.
- No admin/moderator roles.
- No anti-fraud.
- No event replay/history.
- No horizontal [Socket.IO](#) scaling.
- No editing of key tournament parameters after creation.
- The full snapshot is minimal and does not replace full tournament history.

Technical Risks

Risk	Comment
In-memory deadlines	After a restart, there can be a gap between losing old timers and bootstrap recovery
Race conditions	Concurrent votes/finalization are possible, partially mitigated by transactions and row locks
Single instance assumption	The MVP is not designed for multiple backend instances
Presence is ephemeral	Redis presence is not domain history
Timeout votes	At the deadline, missing votes are created for all eligible participants, including offline users
Minimal recovery snapshot	<code>GET /tournaments/:id/full</code> does not contain the full history of rounds/submissions/results
No event replay	The client must perform REST recovery after reconnecting

Deltas Between Requirements and Current Code

`GET /api/v1/tournaments` and `GET /api/v1/tournaments/:id`

In `docs/mvp-requirements.md`, these endpoints are mentioned as not implemented in section 7.2.

In the current code, they are implemented in `TournamentControllerV1`:

- `GET /api/v1/tournaments` - paginated draft tournament list;
- `GET /api/v1/tournaments/:id` - draft tournament details.

In this wiki documentation, they are described as current implemented endpoints.

BullMQ

The requirements say that BullMQ/cron-based phase orchestration is not part of the MVP.

The current code does not use BullMQ as a phase scheduler, but it does use BullMQ for realtime event delivery:

- `TournamentEventsService` enqueues jobs;
- `TournamentEventsProcessor` consumes jobs;
- queue name: `tournaments`.

Therefore, the architecture page lists BullMQ as part of event delivery, while deadlines are described separately as in-memory.

Room Access and Draft Multi-Participation

The business policy allows participation in multiple draft tournaments.

The current `TournamentRoomAccessService.canJoin` additionally checks for unfinished participation elsewhere. This can conflict with the draft multi-participation policy: a user can be a participant in multiple draft tournaments, but WebSocket join can be rejected because of another unfinished participation.

This should be clarified before frontend integration.

APP_CORS_ORIGINS

The `AGENTS.md` environment variable list did not include `APP_CORS_ORIGINS`.

In the current `environment.ts`, this variable is required. It has been added to the wiki documentation as required.

Items to Clarify Before Production

- Whether a distributed `Socket.IO` adapter is needed.
- Whether in-memory deadline registries should be replaced with a durable scheduler.
- Whether the full snapshot should include history of rounds/submissions/results.
- Whether room access should allow connecting to multiple draft tournaments.
- Whether event history should be stored for late join/replay.
- Whether timeout votes should account only for online users or all participants.